

LegalEase: AI-Powered Legal Document Generator

Naan Mudhalvan Project Documentation

Sub Topic: Implement the FastAPI Backend

This project section explains the design and implementation of a FastAPI backend for LegalEase. The backend receives legal-document requirements, validates user data, coordinates document-generation services, stores metadata, and provides secure document downloads.

Project Goal: Build a modular, reliable and secure REST API that connects the LegalEase frontend with document-generation functionality.

2. Introduction

LegalEase is an AI-powered legal document generator designed to simplify the preparation of commonly required legal-document drafts. The application accepts structured information from a user and converts it into a properly organized document through backend services.

The FastAPI backend is the central communication layer between the user interface, database, document templates, AI-assisted generation component, and file-storage system. It exposes REST endpoints that can be consumed by web or mobile clients.

FastAPI is suitable because it supports modern Python development, automatic request validation, asynchronous programming, and automatically generated OpenAPI documentation. The implementation also demonstrates practical backend-development skills relevant to a real-world software project.

Main objective: provide a clean API through which users can submit document requirements, generate a draft, store the result, and securely download the generated document.

3. Backend Requirements

The backend requires Python and a virtual environment. Recommended libraries include FastAPI, Uvicorn, Pydantic, SQLAlchemy, a database driver, python-multipart for upload features when required, and a document-generation library for PDF or DOCX output.

```
python -m venv venv
# Windows: venv\Scripts\activate
# Linux/macOS: source venv/bin/activate
pip install fastapi uvicorn[standard] pydantic sqlalchemy python-multipart
```

Dependencies should be recorded in requirements.txt so that the same environment can be reproduced. Configuration such as database URLs, secret keys and external AI credentials should be supplied through environment variables rather than hard-coded in source code.

Development should also include Git version control and automated tests. A clear README should document installation, environment setup, API execution and testing instructions.

4. Project Architecture

LegalEase should use a modular architecture. API routes handle HTTP requests, Pydantic schemas validate data, services contain business logic, models represent database records, and core modules manage configuration and security.

```
LegalEase/
├── app/
│   ├── main.py
│   ├── api/routes/
│   │   ├── auth.py
│   │   ├── documents.py
│   │   └── health.py
│   ├── schemas/document.py
│   ├── models/document.py
│   ├── services/document_service.py
│   ├── services/ai_service.py
│   ├── db/database.py
│   └── core/config.py
├── tests/
├── requirements.txt
└── README.md
```

This separation prevents route handlers from becoming overloaded. It also allows individual components to be tested independently and makes future maintenance easier.

The architecture can later support PostgreSQL, cloud storage, asynchronous processing, authentication, multilingual generation and additional document types without redesigning the complete application.

5. Creating the FastAPI Application

The main application file creates the FastAPI instance and registers the API routes. A health endpoint is useful for checking whether the backend is available.

```
from fastapi import FastAPI

app = FastAPI(
    title='LegalEase API',
    version='1.0.0',
    description='Backend API for AI-powered legal document generation'
)

@app.get('/health')
def health_check():
    return {'status': 'ok', 'service': 'LegalEase API'}
```

The application can be started during development with Uvicorn. FastAPI automatically exposes interactive API documentation, which makes the backend easy to demonstrate and test.

```
uvicorn app.main:app --reload
```

For production, development reload mode should not be used. The deployment should use suitable process management, HTTPS, environment configuration, logging and monitoring.

6. API Design for Legal Documents

The main resource in LegalEase is the legal document. REST-style endpoints can be used to create, retrieve, update, delete and download documents.

Method	Endpoint	Purpose
POST	/api/v1/documents	Create or generate a document
GET	/api/v1/documents/{id}	Retrieve document metadata
PUT	/api/v1/documents/{id}	Update document data
DELETE	/api/v1/documents/{id}	Delete a document
GET	/api/v1/documents/{id}/download	Download generated file
GET	/health	Check service health

API versioning with /api/v1 makes future changes easier. Consistent HTTP status codes should be used for success, validation errors, authentication failures, missing resources and unexpected server errors.

7. Pydantic Request and Response Models

Pydantic models define the expected structure of JSON requests. Validation at the API boundary prevents incomplete or incorrectly typed data from entering the business layer.

```
from pydantic import BaseModel, Field

class DocumentCreate(BaseModel):
    document_type: str = Field(min_length=2, max_length=100)
    client_name: str = Field(min_length=2, max_length=150)
    purpose: str = Field(min_length=5, max_length=1000)
    jurisdiction: str = Field(min_length=2, max_length=100)
    additional_details: str | None = None

class DocumentResponse(BaseModel):
    id: int
    document_type: str
    status: str
```

Schemas can be extended with addresses, parties, dates, clauses, language and output-format fields. Validation rules should match the actual project requirements.

Response models are equally important because they prevent accidental exposure of internal database fields. Only data needed by the frontend should be returned.

8. Implementing the Document Endpoint

The document route receives validated data and passes it to the service layer. Keeping generation logic outside the route makes the API easier to test and maintain.

```
from fastapi import APIRouter
from app.schemas.document import DocumentCreate
from app.services.document_service import generate_document

router = APIRouter(prefix='/api/v1/documents', tags=['documents'])

@router.post('', status_code=201)
def create_document(payload: DocumentCreate):
    return generate_document(payload)
```

The service can select a template, normalize input, apply business rules, optionally call an AI component, render the output file, store metadata and return the generated document identifier.

If generation becomes slow, the project can later introduce background tasks or a task queue. A synchronous implementation is sufficient when generation is fast for the academic demonstration.

9. Service Layer and AI Integration

The service layer contains the core business logic. It should transform validated user input into a safe, structured document without depending heavily on HTTP-specific objects.

```
def generate_document(payload):
    template = select_template(payload.document_type)
    normalized = normalize_fields(payload)
    draft = build_draft(template, normalized)
    final_text = review_and_format(draft)
    file_path = render_document(final_text)
    return {'status': 'generated', 'file_path': file_path}
```

AI can assist with drafting, rewriting or organizing information, while templates and deterministic validation remain under application control. Generated legal content should be treated as a draft requiring appropriate review before real-world use.

Sensitive information should be minimized when sent to external AI services. Prompt construction should be isolated in the AI service so that it can be tested and changed without modifying API routes.

10. Database Integration

A database can store users, document requests, document metadata, timestamps, status values and references to generated files. SQLite is convenient for a prototype, while PostgreSQL is suitable for a multi-user deployment.

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

DATABASE_URL = 'sqlite:///./legalease.db'
engine = create_engine(DATABASE_URL, connect_args={'check_same_thread': False})
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)
```

A document table may contain id, user_id, document_type, input_data, generated_file, status, created_at and updated_at. Transactions should be used to avoid partially saved records.

Database sessions should be managed through FastAPI dependencies. This creates predictable connection handling and simplifies testing.

11. Authentication and Authorization

If LegalEase stores personal document information, authentication is important. A login endpoint can verify credentials and issue a signed access token.

```
@router.get('/api/v1/me')
def current_user(user=Depends(get_current_user)):
    return {'id': user.id, 'email': user.email}
```

Authorization must be checked for every document operation. A user should be able to access only documents they are permitted to view or modify.

Passwords must never be stored as plain text. Password-hashing libraries should be used, and token-signing keys and other secrets should be kept in secure environment configuration.

The project can later add roles such as user, reviewer and administrator, with permissions appropriate to each role.

12. Error Handling and Validation

A reliable API returns clear errors instead of exposing internal stack traces. FastAPI's `HTTPException` can be used for expected failures, while global exception handlers can standardize error responses.

```
if not document:
    raise HTTPException(status_code=404, detail='Document not found')
```

Validation errors should identify the affected field and explain what needs correction. Business-rule errors should be separated from infrastructure failures.

Unexpected errors should be logged internally and returned as a generic server error. Logs should contain enough information for debugging without unnecessarily storing confidential legal-document content.

A consistent error format improves frontend integration and makes automated tests predictable.

13. File Generation and Download

LegalEase can generate PDF or DOCX files. The service creates the file and stores a controlled reference. An authorized download endpoint then returns the file to the user.

```
from fastapi.responses import FileResponse

@router.get('/{document_id}/download')
def download_document(document_id: int):
    path = get_authorized_file_path(document_id)
    if not path:
        raise HTTPException(status_code=404, detail='File not found')
    return FileResponse(path, filename='LegalEase_Document.pdf')
```

File paths should never be taken directly from untrusted input. Files should use generated names or database identifiers, and authorization must be checked before returning them.

For larger deployments, object storage can replace local disk. The API can then stream the object or provide a controlled download mechanism.

14. API Documentation and Testing

FastAPI generates interactive API documentation from endpoint definitions. This is useful for project evaluation because reviewers can execute requests and inspect request and response schemas.

Testing should cover both successful and failure scenarios. Unit tests validate service functions, while API tests verify endpoints, status codes and response bodies.

```
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_health():
    response = client.get('/health')
    assert response.status_code == 200
    assert response.json()['status'] == 'ok'
```

Additional tests should cover invalid fields, unauthorized access, missing documents, successful generation and download behavior. Automated tests make future changes safer.

15. Security Considerations

Legal documents may contain names, addresses, identification information and agreements. Security must therefore be considered throughout the backend.

Important controls include HTTPS, authentication, authorization, secure password hashing, secret management, input validation, safe file handling, rate limiting where appropriate, dependency updates, database backups and careful logging.

AI prompts and outputs should follow the application's privacy design. Unnecessary personal information should not be sent to external services. The project should document what information is shared and for what purpose.

The backend should also address injection, unsafe file paths, excessive request sizes and unauthorized resource access. Security testing should be completed before public deployment.

16. Environment Configuration

Development and production configurations should be separated. Database URLs, secret keys, AI credentials and allowed origins should be loaded from environment variables.

```
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    database_url: str
    secret_key: str
    ai_api_key: str | None = None

settings = Settings()
```

A local `.env` file can be used during development but should not be committed to a public repository. A `.env.example` file can document required variable names without containing real secrets.

Configuration should be validated at startup so deployment problems are detected early. Logs must never print secret values.

17. Running and Deploying the Backend

During development, Uvicorn can run the FastAPI service locally. Before deployment, the application should be tested in an environment similar to production.

```
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

A production deployment can place the FastAPI service behind a reverse proxy and HTTPS layer. Docker can make deployment more reproducible. Database, file storage and secret management should be treated as separate infrastructure concerns.

A deployment checklist includes installing dependencies, configuring environment variables, applying database migrations, starting the server, enabling HTTPS, configuring logs, creating backups and testing the health endpoint.

18. End-to-End Workflow

The workflow begins when the frontend sends a POST request containing the selected document type and user-provided fields. FastAPI validates the request using the Pydantic schema.

The route calls the document service. The service selects a template, normalizes values, applies deterministic rules and optionally invokes the AI component. The resulting content is checked and rendered into the requested output format.

Metadata is stored in the database. The API returns an identifier and status. When the user selects download, the backend authenticates the request, checks authorization and returns the generated file.

```
Frontend → POST /documents
↓
Pydantic validation
↓
Document service
↓
Template + optional AI assistance
↓
Validation / formatting
↓
File generation + database record
↓
JSON response
↓
Authorized download
```

This workflow provides a clear demonstration of how the frontend and backend cooperate to produce a document.

19. Expected Results and Evaluation

After implementation, LegalEase should provide a working API through which a client can submit document requirements, receive validation feedback, generate a structured draft, store document metadata and download the resulting file.

The implementation demonstrates Python backend development, REST API design, schema validation, database connectivity, modular architecture, authentication concepts, error handling, testing, file processing and deployment planning.

Suggested demonstration sequence:

1. Start the FastAPI server.
2. Open interactive API documentation.
3. Call the health endpoint.
4. Submit a valid document request.
5. Demonstrate invalid-input validation.
6. Display generated document metadata.
7. Download the generated file.
8. Demonstrate authorization.
9. Run automated tests.

Future enhancements can include multilingual generation, document history, reviewer approval, analytics, cloud storage, asynchronous generation and stronger audit logging.

20. Conclusion

Implementing the FastAPI backend establishes the technical foundation of LegalEase: AI-Powered Legal Document Generator. The backend transforms the project concept into a structured service that communicates with the frontend, validates user information, executes document-generation logic, stores records and provides controlled downloads.

The modular approach makes the project suitable for academic demonstration and future development. FastAPI provides the API layer, Pydantic provides validation, the service layer contains business logic, and the database layer provides persistence.

For the Naan Mudhalvan project, this implementation demonstrates how a real-world application can be developed from requirements through architecture, coding, testing and deployment planning.

Final Project Statement: LegalEase combines a guided user experience, reusable templates, structured backend APIs and AI-assisted drafting to make legal-document preparation more organized and accessible. The FastAPI backend provides the scalable foundation for connecting these components into a complete application.

— End of 20-Page Project Section —